

《软件技术基础课程设计》

实验项目：2-2 团队分析

院　　系：计算机科学与技术系

学　　期：2024 年第 1 学期

班　　级：计算机科学与技术 2 班

学　　号：226001624

姓　　名：顾凯琪

指导教师：叶绪伦

日期：2024 年 12 月 13 日

实验二名称：团队综合编程实验（报告+答辩/4-5 人）

任务二-2 按要求阅读和分析

OJ 网站高并发场景下的性能优化		
在使用 OJ 网站的时候，有时候会出现这样的问题：当开始上课的时候，经常会有多个班级登录网站或刷题，此时会出现网页卡顿或 pending 刷屏等情况。请问可以应该采用哪些方法，以求较完善的解决该问题？		
学号	姓名	任务分配
组长	赖诗亮	总体架构及撰写、文献阅读　33%
成员 1	崔智英	案例分析、文献阅读　33%
成员 2	顾凯琪	理论分析、文献阅读　33%

一．引言

在高度信息化的时代背景下，在线 OJ 平台已经成为现代编程教学的重要工具。通过提供实时编程练习和自动化评测功能，OJ 平台极大地提升了教学效率。学生可以通过平台参与丰富的编程比赛、课后练习和团队项目，这使得教学方式更加多样化。

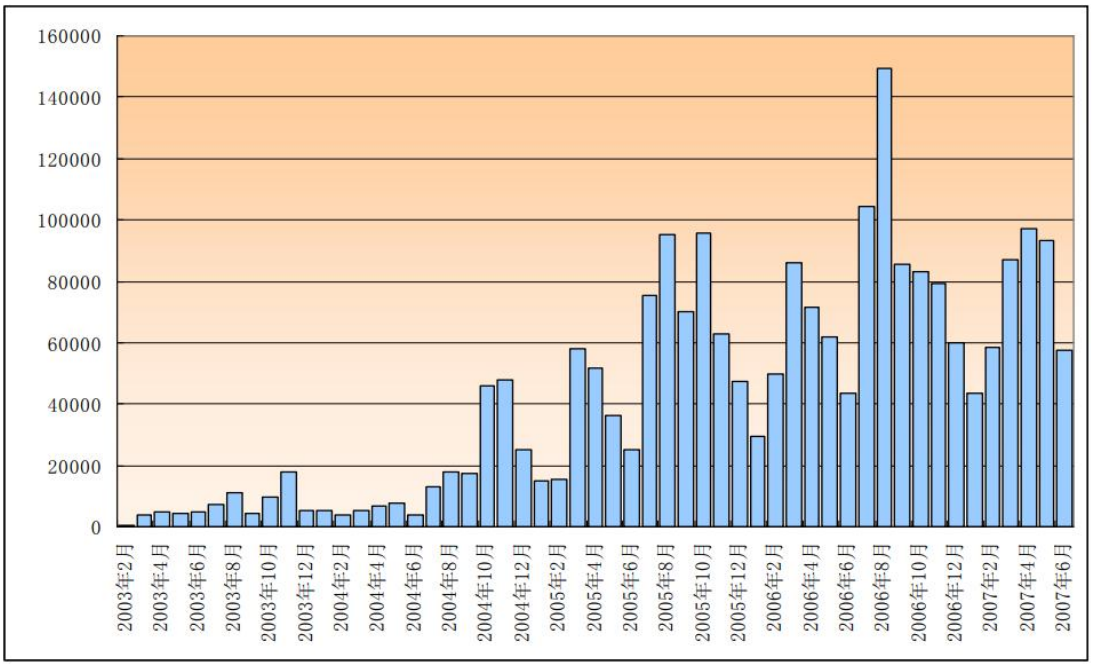


图 1 2003-2007 年北京大学 POJ 系统使用情况

然而，随着平台用户规模的不断扩大以及多人同时使用的频率增加，OJ 平台的性能瓶颈问题日益突出。特别是在多个班级同时进行在线授课或刷题的场景下，网站性能问题如页面卡顿和请求 Pending 变得尤为突出，严重的情况下甚至

出现网页奔溃的现象。这些问题不仅严重影响学生的学习体验，也可能导致课程计划中断，对教学质量造成不良影响。

二．问题研究

针对于当前的众多 OJ 系统所遇到的网页卡顿以及请求 Pending 等问题，这不仅存在着历史遗留问题，同时也是技术以及资源的限制。

2. 1 开发与维护不足

许多校园 OJ 系统由学生开发并维护，这些系统在设计之初往往更注重功能的实现，而缺乏对长期扩展性和性能需求的考虑。随着时间推移，学生开发者的毕业导致运维工作无人接手，系统维护变得更加困难，许多问题得不到及时解决。例如，系统中的漏洞未被修复，性能优化停滞不前，甚至部分功能因环境变化而出现异常。加之这些系统普遍缺乏完善的技术文档和交接机制，进一步加剧了问题的积累，导致校园 OJ 在面对高并发使用场景时难以维持稳定的运行。

宁波大学的 nbuoj 系统是 10 级以来的 ACM 集训队老队员进行开发的。而随着集训队人员的更迭，在学长毕业毕业后的技术传承困难，nbuoj 系统的维护并没有及时跟进，编译器和系统更新滞后。所以对于很多新的语法规则，以及相应的网站性能已无法满足当前教学需求。

2. 2 硬件以及网络资源限制

大部分学校的经费往往是有限的，得综合考虑各专业的经费分配。因此，学校 OJ 系统所使用的服务器通常是成本有限的通用硬件，而非高性能的专业服务器。这种选择虽然降低了初期成本，但在高并发场景下容易出现资源瓶颈。

此外，许多校园网络的稳定性和带宽也难以满足需求，甚至频繁出现断网等问题。例如，在校园网质量欠佳的情况下，即使服务器性能足够，网络资源的限制也会导致访问延迟和页面加载缓慢。这种情况下，OJ 平台的性能优化只能局限于硬件条件的制约。

2. 3 高并发环境下的架构不足

许多 OJ 平台在架构设计上未充分考虑高并发场景，导致大量用户同时访问时，系统压力剧增，常引发请求阻塞和服务中断。单一服务器架构由于缺乏动态资源调度能力，在流量峰值超过其承载能力时，性能迅速下降。与此同时，数据库设计未能针对高并发场景进行优化，例如缺少读写分离机制或合理的分布式数据库方案，这使得频繁的查询和写入请求进一步加剧了系统瓶颈。不少 OJ 平台未部署有效的缓存机制，无法应对高频次的重复查询，增加了服务器负载压力。

因此，通过架构改进来缓解高并发压力已成为 OJ 平台优化的重中之重。

三．高并发场景下的性能优化

在上述所提及的问题当中，本质上的问题是因为大量的用户同时访问，对于系统的并发度要求高。如何解决高并发场景下的性能优化问题，是当且解决 OJ 系统网页卡顿或 pending 刷屏等情况的核心之处。

表 1 北京大学 POJ 系统 24 小时访问情况

时间(时）	网页数	文件数	时间(时）	网页数	文件数
0	74484	281053	12	80315	322324
1	45284	191141	13	89017	359951
2	29097	133378	14	93601	386418
3	18730	82053	15	106054	436564
4	13805	69721	16	109587	453076
5	12307	61924	17	102374	407442
6	12524	66317	18	87020	347633
7	16048	79804	19	92324	377397
8	31353	140781	20	96225	377040
9	58499	232665	21	97100	382176
10	70570	279532	22	95994	370549
11	75053	309833	23	76625	313256

在高并发的场景下，处理的常见方法有负载均衡、异步处理、消息队列、数据优化等方法。其中，我们主要对于负载均衡、消息队列以及异步处理这三种方法。

3. 1 负载均衡

负载均衡是解决 OJ 系统高并发问题的关键技术之一，通过将用户请求均匀分配到多台服务器上，减少单台服务器的负载压力，从而提高系统的整体处理能力。针对 OJ 系统，负载均衡的应用可以有效缓解同时在线用户数量激增时的系统压力，尤其是在比赛和集体练习场景下表现尤为重要。负载均衡的实现方式主要包括硬件负载均衡和软件负载均衡两种：

1. **硬件负载均衡**：通过专用硬件设备（如 F5 负载均衡器）实现流量分发，其优点在于性能稳定、高吞吐量，适合大规模、高并发的应用场景。但在 OJ 系统中，由于学校资源有限，硬件负载均衡的使用成本较高，因此更适合在有充足预算支持的大型 OJ 平台中部署。硬件负载均衡通常还具备健康检查功能，可以定期检测评测服务器的运行状态，确保将流量分发到可用的服务器上，从而避免因单点故障导致的评测服务中断。
2. **软件负载均衡**：如 Nginx 和 HAProxy 等，通过算法（轮询、最小连接数等）将用户请求分配到不同的服务器上。这种方式成本低、灵活性高，是 OJ 系统中较为常见的实现方式。软件负载均衡不仅能够动态调整分配策略，适应评测服务器的实时负载情况，还支持额外功能，如 SSL 卸载用于加快 HTTPS 访问速度，压缩功能减少网络传输流量，以及缓存机制来提升静态资源的访问效率。例如，OJ 系统可以利用 Nginx 缓存常见的题目描述页面，减少对后端服务器的压力。

此外，全局负载均衡结合本地负载均衡（如 DNS 解析与反向代理的配合），能够实现多层次的分流策略。例如，通过基于地理位置的负载均衡（GeoDNS），将用户流量导向最近的服务器节点，减少网络延迟并提升评测的响应速度。同时，在流量突发场景下，例如比赛开始时大量用户同时提交代码，结合弹性扩展机制，

动态增加评测服务器的实例数量，从而提升整个 OJ 系统的负载处理能力。这种策略能够在保障性能的同时，最大化资源利用率，为学生提供更流畅的使用体验。

3.2 异步处理

异步处理通过分离用户请求和耗时操作，降低用户请求的响应延迟，从而提升系统的并发能力。在 OJ 系统中，异步处理的典型应用包括评测任务的后台处理和非实时数据的批量处理。

1. **消息队列的结合**：异步处理通常借助消息队列（如 RabbitMQ、Kafka）实现，将任务放入队列，由后台消费者异步消费。这种方式解耦了前台请求和后台任务的处理流程，避免了前端阻塞。例如，在 OJ 系统中，用户提交的代码会首先进入消息队列，随后由后台判题服务逐一处理，这种模式能够有效避免提交高峰期导致的系统崩溃。同时，为了提升系统的可靠性，可以利用消息队列的确认机制，确保任务不会因为网络或系统故障而丢失。
2. **事件驱动架构**：通过事件驱动的方式（如 Node.js 的事件循环机制或 Java 的 CompletableFuture），能够高效地处理非阻塞 I/O 操作。在 OJ 系统中，提交评测模块是典型的应用场景，用户的代码提交作为事件触发后续的评测任务。事件驱动架构可以避免线程资源浪费，通过非阻塞的方式确保每个任务能够高效执行。此外，这种架构还便于扩展，例如在系统中增加通知功能（如评测完成后自动发送结果通知）时，仅需增加对应的事件监听器，而无需修改现有代码。
3. **任务优先级管理**：在高并发场景下，任务堆积可能导致延迟，因此需要对任务进行优先级划分。例如，在 OJ 系统中，实时评测任务（如比赛中的提交）可以设置为高优先级，而普通练习任务则设置为低优先级，以确保关键任务能够得到及时处理。此外，通过动态调整优先级（如根据任务队列的长度或用户的紧急需求），可以进一步优化系统的响应速度。这种优先级管理机制可以结合消息队列实现，通过不同的队列或标签来标记任务的优先级，后台消费者根据优先级策略分配资源，保证系统的稳定性和高效性。

3.3 消息队列

消息队列是一种高效的任务调度和数据传输方式，在高并发场景下可以作为缓冲层，用于削峰填谷和系统解耦。在 OJ 系统中，消息队列的引入不仅解决了高并发问题，还在任务管理和资源调度上发挥了重要作用。

1. **削峰填谷**：在高并发请求涌入时，将请求写入队列，并由消费者以稳定的速度读取和处理，避免短时间内对后端系统造成过大的压力。例如，在 OJ 系统中，用户提交的代码会被放入消息队列，由评测模块逐步取出进行处理。这样，即使在比赛高峰期，系统也能通过控制消费者的消费速度，平滑地处理评测请求，避免出现系统过载的情况。
2. **分布式消息队列**：如 Kafka，通过分区和副本机制支持高吞吐量和高容错能力。在 OJ 系统中，分布式消息队列的应用可以进一步提升评测任务的并行处理能力。例如，将提交的任务按照题目类型或用户组进行分区，可以让不同的评测节点专注于特定的任务类型，从而提高整体处理效率。同时，Kafka 的副本机制还能够确保任务数据的安全性，即使部分节点出现故障，评测任务依然能够顺利完成。
3. **数据一致性与事务支持**：在复杂业务场景下，需要确保消息处理的幂等性和一致性。例如，在 OJ 系统中，判题模块可能会因为网络问题导致评测

结果未能及时反馈给用户。通过 Kafka 的事务性 API，可以确保评测结果的生成和存储是一个完整的事务，避免因重复消费或失败处理导致的评测数据不一致问题。此外，幂等性机制可以确保同一份代码多次提交只会触发一次评测任务，从而避免资源浪费。

4. **延迟队列**：通过设置消息的延迟时间，可以处理需要延时触发的任务。例如，在 OJ 系统中，某些定时任务（如超时提交判定或比赛结束后的自动结果统计）可以通过延迟队列实现。延迟队列的引入使得这些任务能够以低延迟、低耦合的方式执行，同时不会干扰实时性要求较高的任务。

此外，消息队列的引入虽然能够极大地提升系统的并发能力，但同时也带来了新的问题。例如，队列堆积可能导致任务处理延迟，而消费者的处理能力不足会进一步加剧这一问题。在 OJ 系统中，可以通过动态扩展消费者数量、调整队列优先级或分布式部署等方式来缓解这些问题。同时，利用监控工具可以实时追踪队列的状态，确保任务能够按时完成并及时反馈给用户。

四．总结与思考

OJ 系统这种高并发常见所导致的网页卡顿或 pending 刷屏等情况，其实存在着很多的优化方式。这些方式都能在一定程度上缓解上述的问题，在面对以学校上课人数为计量单位的场景下，能够有效解决上述问题。但是，由于服务器的性能较低以及维护人员不足等限制的情况下，我校 OJ 系统长期未能够进行更新。对于我校 OJ 系统而言，现状的主要瓶颈在于硬件性能低下与长期缺乏维护所导致的技术债务累积。

因此，希望上述建议能够为未来 OJ 系统的重构提供一定的帮助。

参考文献

[1] 吴舜歆,梁宝兰.Online Judge 的设计与实现[J].福建电脑,2012,28(02):166-167.

[2] 黄洪波.大规模编程题在线评判技术研究[D].华南农业大学,2016.

[3] 张平,李慧春.高并发性能问题优化方案研究[J].信息技术与信息化,2023,(11):166-169.

[4] Y. Feng, S. Chen, and W. Zhuang, *High concurrency transaction processing method involves obtaining the current transaction in the high-concurrency transaction queue where balance direction restriction type determines the trading account corresponding to current transaction*, CN111984664-A; CN111984664-B,to Bank China Co Ltd; Bank China Ltd.

[5] C. Li, and L. Qin, *Network security information data real-time analysis monitoring method, involves generating defence strategy according to high concurrency identification result, malicious identification result and high concurrency prediction result*, CN118972127-A,to Qin L.

[6] Z. Liu, C. Feng, and F. Jiang, *Method for optimizing high concurrency platform data queue, involves obtaining optimal network strategy by optimizing set RLIQM controller, and data queue of high-concurrency platform is optimized*, CN115801681-A,to Xiamen Yilian Shidai Technology Co Ltd.

[7] W. Wang, H. Shao, and H. Wang, *High concurrency based micro-service frame realizing method, involves generating single service request, and determining middleware optimization*

parameter based on influencing performance of high concurrency distributed system, CN110569060-A,to Shandong Inspur Common Software Informat.